

Phương pháp chia đôi và các bài toán thống kê

Li Rui

Nguồn gốc: Binary method and statistics problems

I0I China candidate team papers / 2002 / Li Rui.pdf

Mục lục

1	Cây đoạn	2
1.1	Tư tưởng xây dựng cây đoạn	2
1.2	Phương pháp xử lý dữ liệu cơ bản	3
1.3	Ưu thế ứng dụng	4
1.4	Chuyển thành thao tác trên điểm	5
2	Một phương pháp tĩnh để giải thống kê động	6
2.1	Nêu bài toán	6
2.2	Xây dựng và ý tưởng cấu trúc dữ liệu	6
2.3	Duy trì cấu trúc dữ liệu này	6
2.4	Phân tích ứng dụng	8
3	Hiện thực thống kê trên cây tìm kiếm nhị phân	8
3.1	Xây cây tìm kiếm nhị phân tĩnh dùng cho thống kê	9
3.2	Phân tích phương pháp thống kê	11
3.3	Một ví dụ phức tạp hơn	12
4	Cây nhị phân ảo	15
4.1	Hình thái hiện thực cây nhị phân ảo	15
4.2	Một ví dụ cụ thể	16
4.3	Tối ưu quy hoạch động cho dãy đơn điệu dài nhất	17
	Kết luận	19
	Tài liệu tham khảo	20
A	Bản trình chiếu kèm theo trong PDF	20
A.1	Giới thiệu	20
A.2	Cây đoạn	20
A.3	Một phương pháp thống kê tĩnh	22

A.4	Hiện thực bằng cây tìm kiếm nhị phân tĩnh	23
A.5	Hiện thực cây bằng phương pháp chia đôi ảo	25
A.6	Ví dụ trong trình chiếu: dãy giảm dài nhất	25

Từ khóa. Cây đoạn; cây nhị phân; phương pháp chia đôi.

Tóm tắt

Ta thường gặp các bài toán thống kê. Đặc điểm của các bài toán này là hình thức thường khá đơn giản: nói chung chỉ cần xử lý dữ liệu trong một phạm vi nhất định và có thể hiện thực bằng các phương pháp cơ bản. Tuy nhiên quy mô xử lý thực tế lại khá lớn, nên thuật toán thô sơ chỉ dẫn đến hiệu quả thấp. Để khắc phục khó khăn ấy, khi làm thống kê ta cần mượn một số công cụ đặc biệt, chẳng hạn các cấu trúc dữ liệu hiệu quả.

Bài viết này chủ yếu giới thiệu cách kết hợp tư tưởng chia để trị với một số cấu trúc dữ liệu, làm cho quá trình thống kê có một khuôn mẫu nhất định để nâng cao hiệu quả. Trước hết bài viết giới thiệu ngắn gọn cơ sở của cây đoạn, một cấu trúc dữ liệu rất thích hợp cho hình học tính toán và cũng có thể mở rộng sang những mặt khác. Sau đó bài viết giới thiệu một phương pháp thống kê đặc biệt liên quan đến IOI 2001. Tiếp theo là một kiểu xây dựng hơi khác với cây đoạn, dưới dạng cây tìm kiếm nhị phân; ta sẽ thấy phương pháp này rất linh hoạt. Xa hơn, ta bỏ qua việc xây dựng tường minh cây này và thực hiện nó một cách “ảo” trên một bảng có thứ tự.

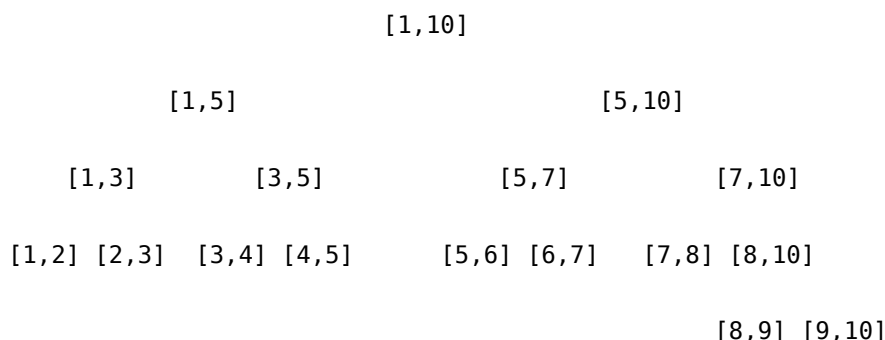
1 Cây đoạn

Trong một lớp bài toán, ta phải thường xuyên xử lý một số đoạn thẳng cố định có thể ánh xạ lên một trục tọa độ, chẳng hạn các đoạn trên trục OX . Vì các đoạn có thể phủ nhau, đôi khi ta cần lấy hợp của các đoạn một cách động, ví dụ lấy tổng độ dài của các khoảng hợp hoặc số khoảng trong hợp. Một đoạn tương ứng với một khoảng, vì vậy cây đoạn cũng có thể gọi là cây khoảng.

1.1 Tư tưởng xây dựng cây đoạn

Cây đoạn xử lý một số đoạn cố định, hay nói cách khác các đoạn này có thể tương ứng với hữu hạn đầu mút cố định. Khi xử lý bài toán, trước tiên ta trừu tượng hóa các đầu mút khoảng, ví dụ có N đầu mút $t_i (1 \leq i \leq N)$. Khi đó với bất kỳ đoạn cần xử lý $[a, b]$, luôn tìm được các chỉ số tương ứng i, j sao cho $t_i = a, t_j = b, 1 \leq i \leq j \leq N$. Phép chuyển đổi này làm cho các khoảng trên cây đoạn được biểu diễn bằng số nguyên; qua ánh xạ, bài toán khoảng thực ban đầu vẫn được xử lý theo cùng cách.

Ví dụ sau là một cây đoạn biểu diễn được khoảng $[1, 10]$:



Cây đoạn là một cây nhị phân; mỗi nút trong cây biểu diễn một khoảng $[a, b]$. Tại mỗi lá có $a + 1 = b$, biểu thị một khoảng sơ cấp. Với mỗi nút trong có $b - a > 1$, nếu cây đoạn gốc $[a, b]$ là $T(a, b)$, thì tiếp tục chia nó thành cây con trái $T(a, \lfloor (a + b)/2 \rfloor)$ và cây con phải $T(\lfloor (a + b)/2 \rfloor, b)$, cho đến khi tách thành khoảng sơ cấp.

Cách xây dựng bằng chia đôi của cây đoạn thực chất dùng phương pháp chia đôi. Cây đoạn là một cây cân bằng, độ sâu của nó là $\lg(b - a)$.

Nếu dùng cấu trúc dữ liệu động để hiện thực cây đoạn, nút có thể được mô tả bằng kiểu dữ liệu sau:

Type

```
Tnode = ^Treenode;
Treenode = record
    B, E: integer;
    Count: integer;
    LeftChild, RightChild: Tnode;
End;
```

Trong đó B và E biểu thị khoảng $[B, E]$; Count là một bộ đếm, thường ghi số đoạn phủ lên khoảng này. LeftChild và RightChild lần lượt là gốc của cây con trái và cây con phải.

Để thuận tiện, ta cũng có thể dùng cấu trúc dữ liệu tĩnh với các mảng $B[]$, $E[]$, $C[]$, $LSON[]$, $RSON[]$. Giả sử gốc của một cây đoạn là v . Khi đó $B[v]$, $E[v]$ là biên của khoảng mà nó biểu diễn; $C[v]$ vẫn dùng làm bộ đếm; $LSON[v]$ và $RSON[v]$ lần lượt biểu thị số hiệu gốc của con trái và con phải.

Cần chú ý rằng đây chỉ là cấu trúc cơ bản của cây đoạn. Khi sử dụng cây đoạn, thường phải thêm vào mỗi nút một số trường dữ liệu đặc biệt và duy trì động các trường đó theo thao tác chèn, xóa đoạn. Điều này tùy từng bài, đồng thời thường là linh hồn của lời giải.

1.2 Phương pháp xử lý dữ liệu cơ bản

Các thao tác cơ bản nhất của cây đoạn là xây dựng, chèn và xóa. Dưới đây lấy cấu trúc dữ liệu tĩnh làm ví dụ.

Xây dựng cây đoạn (a, b) . Đặt một biến toàn cục n để ghi tổng số nút đã dùng; ban đầu $n = 0$.

```
procedure BUILD(a, b)
begin
    n <- n + 1
    v <- n
    B[v] <- a
    E[v] <- b
    C[v] <- 0
    if b - a > 1 then
    begin
        LSON[v] <- n + 1
        BUILD(a, floor((a + b) / 2))
        RSON[v] <- n + 1
        BUILD(floor((a + b) / 2), b)
    end
end
```

Chèn khoảng $[c, d]$ vào cây đoạn $T(a, b)$. Giả sử số hiệu gốc của $T(a, b)$ là v .

```

procedure INSERT(c, d; v)
begin
    if c <= B[v] and E[v] <= d then C[v] <- C[v] + 1
    else
        begin
            if c < floor((B[v] + E[v]) / 2) then INSERT(c, d; LSON[v])
            if d > floor((B[v] + E[v]) / 2) then INSERT(c, d; RSON[v])
        end
    end
end

```

Giải thích thuật toán: nếu $[c, d]$ phủ hoàn toàn đoạn hiện tại, rõ ràng cơ sở trên nút này, tức số đoạn phủ, tăng thêm 1. Ngược lại, nếu $[c, d]$ không vượt qua trung điểm của khoảng thì chỉ chèn vào cây con trái hoặc cây con phải. Nếu có vượt qua trung điểm thì phải chèn vào cả hai cây con. Quan sát đường đi chèn, một khoảng cần chèn có thể “vượt qua” tại một nút nào đó; sau đó trên hai cây con đều tiếp tục đi xuống, nhưng sự vượt qua như vậy không thể xảy ra nhiều lần. Độ phức tạp thời gian để chèn một khoảng là $O(\log n)$.

Xóa một khoảng khỏi cây đoạn gần như hoàn toàn tương tự chèn:

```

procedure DELETE(c, d; v)
begin
    if c <= B[v] and E[v] <= d then C[v] <- C[v] - 1
    else
        begin
            if c < floor((B[v] + E[v]) / 2) then DELETE(c, d; LSON[v])
            if d > floor((B[v] + E[v]) / 2) then DELETE(c, d; RSON[v])
        end
    end
end

```

Đặc biệt chú ý: chỉ những khoảng đã từng được chèn mới có thể bị xóa. Như vậy việc duy trì cây đoạn mới đúng. Ví dụ trên cây đoạn đã nêu, không thể chèn $[1, 10]$ rồi xóa $[2, 3]$; hiển nhiên làm vậy không cho kết quả đúng.

Tác dụng của cây đoạn chủ yếu nằm ở chỗ có thể duy trì động một số đặc trưng. Chẳng hạn muốn lấy độ dài hợp các đoạn trên cây đoạn, ta thêm một trường dữ liệu $M[v]$ và xét:

$$M[v] = \begin{cases} E[v] - B[v], & C[v] > 0, \\ 0, & C[v] = 0 \text{ và } v \text{ là lá,} \\ M[LSON[v]] + M[RSON[v]], & C[v] = 0 \text{ và } v \text{ là nút trong.} \end{cases}$$

Chỉ cần mỗi lần chèn hoặc xóa khoảng, cập nhật giá trị M tại các nút được thăm, tạm gọi thao tác này là UPDATA, thì có thể vừa chèn, xóa vừa duy trì tốt M . Khi cần độ dài hợp của toàn cây đoạn, chỉ cần lấy $M[ROOT]$. Điều này rất hữu ích trong nhiều bài toán duy trì động: chi phí duy trì cho mỗi thao tác chỉ là $\log n$.

Tương tự còn có các bài toán như tìm số khoảng trong hợp. Ở đây ta không liệt kê sâu hơn.

1.3 Ưu thế ứng dụng

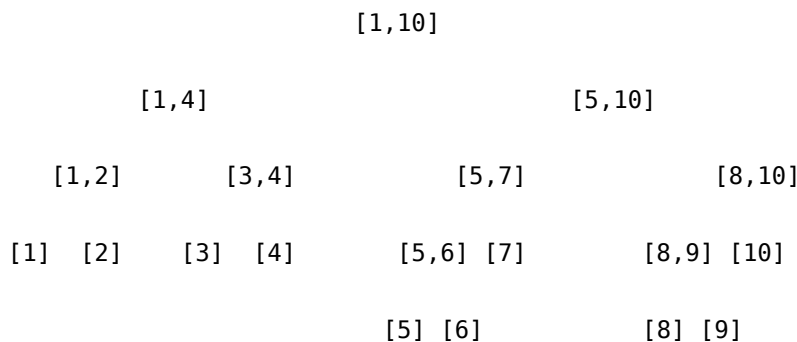
Cách xây dựng cây đoạn chủ yếu phục vụ xử lý đoạn và khoảng, nên thường được dùng trong các bài toán hình học tính toán. Chẳng hạn khi xử lý một nhóm hình chữ nhật, có thể dùng nó để tìm chu vi biên

hoặc diện tích sau khi lấy hợp các hình chữ nhật, hiệu quả hơn rời rạc hóa thông thường. Các ứng dụng này có thể tìm thấy trong tài liệu liên quan; ở đây không đi sâu.

1.4 Chuyển thành thao tác trên điểm

Cây đoạn xử lý các bài toán khoảng đoạn; một số bài toán thống kê lại xử lý điểm. Nhưng điểm cũng có thể hiểu là một khoảng đặc biệt. Khi đó ta thường biến đổi cách xây dựng cây đoạn, tức chuyển thành một cấu trúc ghi nhận điểm.

Biến đổi như sau: tách các khoảng sơ cấp trên cây đoạn thành các điểm cụ thể để đếm. Nghĩa là không còn những khoảng kiểu $(a, a + 1)$; mỗi điểm được tách ra thành a và $a + 1$. Đồng thời khi chia theo trung điểm của khoảng, mỗi điểm hoặc rơi vào cây con trái, hoặc rơi vào cây con phải:



Khi đó trường đếm $C[v]$, vốn dùng để ghi cơ sở số đoạn, có thể dùng để tính số điểm rơi vào một khoảng nhất định. Đường tìm kiếm ban đầu cũng thay đổi: không còn tình huống “vượt qua”. Đồng thời khi chèn mỗi điểm đều phải đi sâu đến lá, vì vậy nói chung đều có độ phức tạp $\log n$.

Một mặt, dùng loại cây đoạn này thuận tiện cho việc đếm. Mặt khác, vì về bản chất nó là cây tìm kiếm nhị phân nên dễ tìm giá trị lớn nhất và nhỏ nhất. Sau đây là một ví dụ tìm lớn nhất và nhỏ nhất.

Ví dụ 1: bài *Promotion* (POI0015).

Tóm tắt đề. Một khách hàng mua sắm trong n ngày $(1 \leq n \leq 5000)$. Mỗi ngày người đó có một số hóa đơn. Sau mỗi ngày mua sắm, từ tất cả hóa đơn trước đó người đó chọn ra hai hóa đơn: một hóa đơn có mệnh giá lớn nhất và một hóa đơn có mệnh giá nhỏ nhất, rồi xóa hai hóa đơn này khỏi hồ sơ. Các hóa đơn còn lại được giữ để tiếp tục thống kê về sau. Dữ liệu vào bảo đảm tổng số hóa đơn trong n ngày không vượt quá 1 000 000, và mệnh giá mỗi hóa đơn là số nguyên từ 1 đến 1 000 000. Bảo đảm mỗi ngày luôn tìm được hai hóa đơn.

Cách giải. Bài này thể hiện rất rõ đặc tính duy trì động: mỗi ngày phải chèn một số hóa đơn có mệnh giá tùy ý, đồng thời còn phải tìm hóa đơn lớn nhất và nhỏ nhất. Ta có thể xây cây đoạn điểm như trên, với phạm vi $[1, 1\,000\,000]$, tức xem mọi mệnh giá hóa đơn là một điểm. Chèn và xóa một hóa đơn là hiển nhiên.

Làm thế nào tìm hóa đơn lớn nhất? Với một cây v , nếu $C[\text{LSON}[v]] > 0$ thì giá trị nhỏ nhất trong cây v chắc chắn nằm ở cây con trái. Tương tự, nếu $C[\text{RSON}[v]] > 0$ thì giá trị lớn nhất nằm ở cây con phải; nếu $C[\text{LSON}[v]] = 0$ thì cả hóa đơn lớn nhất và nhỏ nhất đều nằm ở cây con phải. Vì thế phép đếm của cây đoạn thực ra đã cung cấp manh mối. Với một mệnh giá cụ thể, đường đi chèn, xóa và tìm kiếm là như nhau, có độ dài bằng độ sâu cây, tức $\log 1\,000\,000 = 20$. Nếu tổng cộng có N hóa đơn, xét giới hạn thì độ phức tạp là $N \cdot 20 + n \cdot 20 \cdot 2$, đơn giản hơn nhiều so với hiện thực bằng sắp xếp thông thường.

Bài này còn có thể dùng một cách khéo hơn: cây đoạn không nhất thiết phải lưu mệnh giá cụ thể của hóa đơn. Vì ta lưu toàn bộ 1 000 000 loại mệnh giá, cây đoạn khá lớn. Có thể dùng bảng băm để lưu số lượng hóa đơn của từng mệnh giá; còn với một hóa đơn cụ thể, chẳng hạn mệnh giá V , ta lưu $V/100$ trên cây đoạn, tức xem 100 mệnh giá liên tiếp là một nhóm. Vì phạm vi của V là $[1, 1\,000\,000]$, cây đoạn chỉ có 10 000 điểm. Khi tìm số lớn nhất, trước tiên tìm nhóm tương ứng trên cây đoạn, rồi tìm trong bảng băm của nhóm này; quy mô tìm kiếm hiển nhiên không vượt quá 100. Vì cây đoạn nhỏ hơn nên độ sâu chỉ khoảng 14, giới hạn độ phức tạp toàn bài là $N \cdot 14 + n \cdot 14 \cdot 100 \cdot 2$, với quy mô đề bài vẫn hiệu quả. Cách làm này tiết kiệm không gian ở một mức độ nhất định, đồng thời nhắc ta rằng cần dùng cây đoạn một cách linh hoạt.

2 Một phương pháp tính để giải thống kê động

2.1 Nêu bài toán

Ví dụ 2: IOI 2001 *Mobiles*. Trong một bảng $N \times N$, ban đầu số trong mỗi ô đều bằng 0. Nay có một số câu hỏi và phép sửa đổi động. Câu hỏi có dạng: tìm tổng mọi phần tử trong một hình chữ nhật con xác định $(x_1, y_1) - (x_2, y_2)$. Phép sửa đổi có dạng: chỉ định một ô (x, y) , rồi cộng hoặc trừ một giá trị xác định A vào phần tử trong ô (x, y) . Yêu cầu trả lời đúng mọi câu hỏi. $1 \leq N \leq 1024$, tổng số câu hỏi và sửa đổi có thể đạt 60000.

Như đã nói trong phần mở đầu, ý nghĩa của loại bài này rất đơn giản, dùng vài vòng lặp nhỏ là có thể giải. Nhưng trước quy mô có thể phải xử lý, ta không thể dùng cách đó, vì hiện thực đơn giản có hiệu quả quá thấp. Một cách giải hoàn hảo cho bài này dùng đến một định nghĩa cấu trúc đặc biệt. Bài toán là hai chiều; chú ý tư tưởng hạ bậc, ta thảo luận bài toán một chiều rồi chỉ cần mở rộng nhẹ.

Bài toán tính tổng dãy một chiều như sau: giả sử các phần tử của dãy lưu trong $a[]$, chỉ số của a là các số nguyên dương $1..n$. Cần cập nhật động giá trị của một $a[x]$, đồng thời cần tìm tổng mọi phần tử từ $a[x_1]$ đến $a[y_1]$. Bài toán này thực chất có phát biểu giống *Mobiles*.

2.2 Xây dựng và ý tưởng cấu trúc dữ liệu

Bài này thật ra có thể giải hiệu quả bằng cây đoạn đã nói ở trên. Ta biết tổng từ $a[x_1]$ đến $a[y_1]$ có thể tính bằng $\text{sum}(1, y_1) - \text{sum}(1, x_1 - 1)$, tức kỹ thuật lấy hiệu hai tổng tiền tố. Dạng $\text{sum}(1, x)$ có thể thu được nhanh trên cây đoạn, và cách sửa giá trị phần tử cũng tương tự. Ở đây không nói chi tiết. Ta muốn xây dựng thêm một hình thức đặc biệt, vì nó hiện thực đơn giản hơn cây đoạn rất nhiều. Đồng thời tư tưởng này cũng rất thú vị và tinh xảo.

Với dãy $a[]$, đặt một mảng C , trong đó

$$C[i] = a[i - 2^k + 1] + \dots + a[i],$$

với k là số lượng bit 0 liên tiếp ở cuối biểu diễn nhị phân của i . Khi đó các thao tác của ta hiển nhiên có liên hệ đặc biệt với mảng C này. Trong mảng ghi nhận ấy, $C[K]$ biểu hiện ra sao? Ví dụ $C[56]$: viết 56 dưới dạng nhị phân được 111000, nên số nhỏ nhất mà $C[56]$ biểu diễn là 110001, tức 49. Vì vậy $C[56]$ là tổng các phần tử từ $a[49]$ đến $a[56]$. Có thể thấy mỗi phần tử của C biểu diễn theo cách không có quy luật cụ thể nhìn từ hệ thập phân; chẳng hạn $C[7]$ chỉ biểu thị giá trị $a[7]$.

2.3 Duy trì cấu trúc dữ liệu này

Có lẽ bạn đã chú ý rằng định nghĩa của C rất kỳ lạ, dường như không nhìn ra quy luật. Sau đây ta nghiên cứu cụ thể tính chất của C , xét cách sửa một phần tử trong đó và cách tìm tổng tiền tố; sau đó ta sẽ thấy công dụng của C rất khéo.

Tính 2^k tương ứng với $C[x]$. Ở đây k là số bit 0 liên tiếp ở cuối của x trong hệ nhị phân. Từ định nghĩa có thể thấy phép tính này được dùng thường xuyên. Có thao tác đơn giản nào để nhận được kết quả không? Ta có thể dùng công thức:

$$2^k = x \text{ and } (x \text{ xor } (x - 1)).$$

Công thức này khéo léo dùng phép toán bit và chỉ cần hai bước tính đơn giản. Chứng minh có thể thu được bằng cách liên hệ cách dùng cụ thể của các phép bit với đặc trưng của x . Trong phần sau, ta gọi công thức này là hàm $\text{LOWBIT}(x)$.

Sửa giá trị của một $a[x]$. Trong bài toán đã nêu, thực chất ta phải giải hai việc: sửa giá trị $a[x]$ và tìm tổng tiền tố. Ta đã mượn C để biểu diễn một số tổng của a , vì vậy lời giải cho hai việc này là cập nhật các lượng liên quan trong C . Khi sửa một $a[x]$, chỉ cần sửa mọi giá trị $C[i]$ có liên quan, tức có chứa $a[x]$. Những $C[i]$ nào có thể chứa $a[x]$? Lấy ví dụ $x = 1001010_2$, quan sát hình thức ta được:

$$\begin{aligned} p_1 &= 1001010, \\ p_2 &= 1001100, \\ p_3 &= 1010000, \\ p_4 &= 1100000, \\ p_5 &= 1000000. \end{aligned}$$

Mỗi p_i ở đây đều có thể chứa x , tức giá trị $C[p_i]$ bất kỳ đều chứa giá trị $a[x]$. Dãy số này có quy luật gì? Có thể thấy:

$$p_1 = x, \quad p_i < p_{i+1}, \quad p_{i+1} = p_i + \text{LOWBIT}(p_i).$$

Quan sát trực tiếp dễ thấy điều này đúng, và cũng dễ chứng minh về mặt lý thuyết. Các số này có bao gồm mọi giá trị cần sửa không? Xét đặc trưng của số nhị phân, có thể thấy với mọi $p_i < Y < p_{i+1}$, $C[Y]$ chứa các giá trị $a[p_i + 1] + \dots + a[Y]$, nên không thể chứa $a[x]$.

Quan sát thêm cách sinh dãy p , thực ra mỗi lần ta cộng vào bit 1 cuối cùng để được số kế tiếp. Vì vậy dãy p chứa nhiều nhất $\lg n$ số, trong đó n là độ dài bảng a , hay cũng là độ dài bảng C . Vì ta chỉ ghi các giá trị $C[1]$ đến $C[n]$, khi số sinh ra trong dãy p lớn hơn n , quá trình không cần tiếp tục. Trong nhiều trường hợp khi sửa $a[x]$, độ dài dãy p nhỏ hơn $\log n$ rất nhiều. Với các n thường gặp, chỉ vài bước là hoàn thành. Sửa một phần tử $a[x]$, làm nó tăng thêm A và trở thành $a[x] + A$, có thể làm như sau:

```
procedure UPDATA(x, A)
begin
  p <- x
  while p <= n do
  begin
    C[p] <- C[p] + A
    p <- p + LOWBIT(p)
  end
end
```

Tính kết quả của một câu hỏi $[x, y]$. Sau đây ta giải quyết việc tìm tổng tiền tố. Theo kinh nghiệm trước đó, chuyển bài toán thành tìm $\text{sum}(1, y) - \text{sum}(1, x - 1)$. Vậy làm thế nào dựa vào giá trị C để tìm $\text{sum}(1, x)$? Dễ thu được quá trình sau:

```

function SUM(x)
begin
  ans <- 0
  p <- x
  while p > 0 do
  begin
    ans <- ans + C[p]
    p <- p - LOWBIT(p)
  end
  return ans
end

```

Quá trình này rất giống UPDATA và dễ hiểu. Độ phức tạp của nó hiển nhiên là $\lg n$. Mỗi lần trả lời một câu hỏi chỉ cần thực hiện SUM hai lần rồi lấy hiệu, nên một câu hỏi cần $2 \log n$ thao tác.

Qua hai bước phân tích trên, ta thấy nhờ định nghĩa khéo léo của C , độ phức tạp của quá trình duy trì mảng và tính tổng đều giảm xuống $\log n$. Đây là kết quả rất đáng mừng và thỏa mãn.

2.4 Phân tích ứng dụng

Với bài toán một chiều đã nêu, hai hàm trên giải quyết dễ dàng. Chú ý bộ nhớ cần tiêu thụ chỉ là một mảng đơn, cách xây dựng đơn giản hơn cây đoạn rất nhiều. Rõ ràng bài này cũng có thể giải bằng cấu trúc cây đoạn ở phần trước.

Chỉ cần mở rộng tốt bài toán một chiều này lên hai chiều là có thể giải bài *Mobiles* của IOI 2001. Mở rộng thế nào? Trong *Mobiles*, ta cần sửa giá trị $a[x, y]$. Mô phỏng lời giải một chiều, có thể định nghĩa

$$C[x, y] = \sum a[i, j],$$

trong đó

$$x - \text{LOWBIT}(x) + 1 \leq i \leq x, \quad y - \text{LOWBIT}(y) + 1 \leq j \leq y.$$

Quá trình sửa và tính tổng cụ thể thực chất là hai vòng lồng nhau của quá trình một chiều. Có thể thấy sau lần mở rộng này, độ phức tạp thuật toán là $\log^2 n$. Về không gian, ta chỉ đơn giản đổi mảng một chiều thành mảng hai chiều, chi phí mở rộng tương đối thấp.

Có thể thử xây cây đoạn hai chiều để giải bài này; độ phức tạp của nó cao hơn rất nhiều so với phương pháp tính này. Trong luật thi IOI 2001, *Mobiles* bị giới hạn bộ nhớ 5 MB. Dùng phương pháp của phần này, chỉ cần 4 MB cho mảng C và một vài biến rời rạc. Nếu dùng cách thô bạo xây cây đoạn ở phần một, nút thất bộ nhớ là điều có thể hình dung.

Phương pháp thống kê đặc biệt này rất có ưu thế cho bài này, đồng thời khá dễ mở rộng lên số chiều cao hơn, điều cây đoạn ở phần trước không sánh được. Nhưng phương pháp của phần này cũng có khuyết điểm: dường như không dễ mở rộng sang những bài toán khác. Nếu nghiên cứu kỹ cây đoạn, ta sẽ biết nó có thể hỗ trợ nhiều bài toán thống kê đặc biệt. Điều này sẽ thể hiện qua thực hành. Sau đây giới thiệu thêm một số phương pháp hiện thực khác.

3 Hiện thực thống kê trên cây tìm kiếm nhị phân

Như đã nói ở trên, sau khi chia trái phải, cây đoạn thực chất có tính chất của cây tìm kiếm nhị phân. Mặt khác, phần trước cũng nói cách xây dựng cây đoạn rất thích hợp để xử lý đoạn; với bài toán điểm, nó có thể được mở rộng ứng dụng, như ví dụ 1, nhưng luôn có cảm giác hơi quá mức cần thiết. Một mặt, trên

cây đoạn phải đặt quá nhiều con trỏ đến cây con trái và cây con phải; mặt khác, nút dùng để biểu diễn khoảng, trong khi xử lý điểm thì không cần giữ lại những khoảng như vậy. Khi một nút trên cây đoạn tách thành hai nửa khoảng, thực ra nó được chia bởi một điểm ở giữa. Vậy trong bài toán thống kê điểm, chỉ cần giữ lại các điểm chia như vậy.

3.1 Xây cây tìm kiếm nhị phân tĩnh dùng cho thống kê

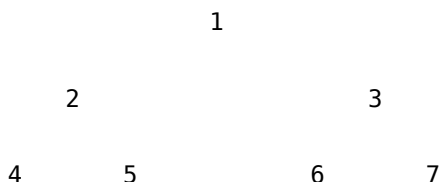
Với bài toán xử lý điểm, chỉ cần xây một cây tìm kiếm nhị phân sao cho mọi điểm cần xử lý đều tìm được nút tương ứng trên cây. Đồng thời nhờ tính chất của cây nhị phân: mọi giá trị ở cây con trái nhỏ hơn gốc, mọi giá trị ở cây con phải lớn hơn gốc, ta kế thừa ưu điểm của cây đoạn. Trước tiên, với mọi điểm cần xử lý, xây một cây tìm kiếm nhị phân có thể thống kê tĩnh làm khuôn mẫu. Ví dụ với tập $\{3, 4, 5, 8, 19, 23, 6\}$, có thể xây một cây thống kê tìm kiếm nhị phân gồm 7 điểm:



Chú ý giá trị ghi trên mỗi nút chính là giá trị điểm tương ứng.

Bước đầu tiên để xây cây thống kê nhị phân là rời rạc hóa mọi tọa độ điểm cần xử lý để tạo thành một ánh xạ đã sắp thứ tự, gọi là ánh xạ X , và giả sử trong đó có n đối tượng khác nhau. Trong ví dụ trên, $X = \{3, 4, 5, 6, 8, 19, 23\}$, $n = 7$.

Bây giờ cần điền các giá trị điểm trong ánh xạ X vào cây để nó có cấu trúc như trên. Ta chọn cấu trúc tĩnh để hỗ trợ cây nhị phân. Đánh số các nút từ trên xuống dưới, từ trái sang phải, và đặt số hiệu gốc là 1. Khi đó các số hiệu tương ứng là:



Cách này rất giống hiện thực heap tĩnh. Với bất kỳ nút có số hiệu i , con trái của nó tự nhiên có số hiệu $i \cdot 2$, con phải có số hiệu $i \cdot 2 + 1$. Bây giờ cần điền ánh xạ X vào mảng V ; $V[1..n]$ cần lưu giá trị điểm ở vị trí tương ứng.

Chú ý kết quả duyệt trung thứ tự cây nhị phân ứng với V chính là ánh xạ trong X , vì vậy có thể xây V bằng đệ quy:

```

p <- 0 {con trỏ trong ánh xạ X}

procedure BUILD(ID: integer) {ID là chỉ số nút V}
begin
  if ID * 2 <= n then BUILD(ID * 2)
  p <- p + 1
  V[ID] <- X[p]
  if ID * 2 + 1 <= n then BUILD(ID * 2 + 1)
end
  
```

{Trong chương trình chính gọi BUILD(1).}

Quá trình này thực hiện duyệt trung thứ tự trên V .

Nếu quen với quá trình duyệt trung thứ tự cây nhị phân, cũng có thể không dùng đệ quy. Chỉ cần bắt đầu từ nút đầu tiên của phép duyệt trung thứ tự, mỗi lần tìm nút kế tiếp của nó. Nút đầu tiên hiển nhiên là nút ngoài cùng bên trái ở tầng dưới cùng của cây nhị phân. Nếu có n nút, trước hết tìm nút đầu tiên bằng quá trình sau:

```
function first: integer
begin
    level <- 1
    tot <- 2
    while tot - 1 < n do
    begin
        level <- level + 1
        tot <- tot * 2
    end
    return tot div 2
end
```

Sau đó gán giá trị cho V bằng quá trình:

```
procedure BUILD
begin
    now <- first
    p <- 0
    for i <- 1 to n do
    begin
        p <- p + 1
        V[now] <- X[p]
        if now * 2 + 1 <= n then
        begin
            now <- now * 2 + 1
            while now * 2 <= n do now <- now * 2
        end
        else
        begin
            while now is odd do now <- now div 2
            now <- now div 2
        end
    end
end
```

Từ cách xây dựng có thể thấy đây là một cây gần đầy, vì vậy cũng là một cây cân bằng, với độ sâu $\log n$.

3.2 Phân tích phương pháp thống kê

Trong cây này, với bất kỳ điểm cần xử lý nào có giá trị $value$, ta dễ dàng tìm được nút tương ứng theo $value$. Chẳng hạn cần duy trì động số lượng điểm: tương tự ví dụ 1, trên mỗi nút đặt một SUM, biểu thị tổng số điểm trên cây con gốc tại nút đó. Ban đầu $SUM[i] = 0$. Chèn một điểm như sau:

```
procedure INSERT(value)
begin
  now <- 1
  repeat
    SUM[now] <- SUM[now] + 1
    if V[now] = value then break
    if V[now] > value then now <- now * 2
    else now <- now * 2 + 1
  until false
end
```

Ta có thể duy trì động SUM trong thời gian $\log n$; quá trình này đồng bộ với việc tìm kiếm $value$.

Việc đặt SUM như trên khá thông thường. Một số cách đặt đặc biệt có tác dụng lớn hơn. Ví dụ đặt LESS tại mỗi nút, biểu thị tổng số điểm có giá trị nhỏ hơn hoặc bằng giá trị gốc, tức tổng số điểm trên gốc và cây con trái. Khi chèn:

```
procedure INSERT1(value)
begin
  now <- 1
  repeat
    if value <= V[now] then LESS[now] <- LESS[now] + 1
    if V[now] = value then break
    if V[now] > value then now <- now * 2
    else now <- now * 2 + 1
  until false
end
```

Quá trình này gần giống quá trình trước. Thực ra $LESS[i] = SUM[i] - SUM[i \cdot 2 + 1]$. Nêu ví dụ này để cho thấy dùng cấu trúc cây tìm kiếm nhị phân thì rất dễ biến đổi theo bài toán cụ thể.

Ta thậm chí có thể biến đổi nó để giải ví dụ 2. Chỉ cần thay đổi nhẹ định nghĩa LESS, cho nó là tổng trọng số của mọi điểm trên gốc và cây con trái. Nếu cần tăng $a[x]$ thêm A , để có:

```
procedure INSERT2(x, A)
begin
  now <- 1
  repeat
    if x <= V[now] then LESS[now] <- LESS[now] + A
    if V[now] = x then break
    if V[now] > x then now <- now * 2
    else now <- now * 2 + 1
  until false
end
```

Tương tự cũng rất thuận tiện. Nếu cần giá trị $SUM(1, x)$, chỉ cần dùng hàm:

```
function SUM(x): longint
begin
    ans <- 0
    now <- 1
    repeat
        if V[now] <= x then ans <- ans + LESS[now]
        if V[now] = x then break
        if V[now] > x then now <- now * 2
        else now <- now * 2 + 1
    until false
    return ans
end
```

Có thể thấy các quá trình này về cơ bản giống nhau. Hiệu quả của cách hiện thực này khi giải ví dụ 2 không thua kém phương pháp ở phần hai, đồng thời tiêu thụ bộ nhớ cũng chỉ là một mảng đơn. Nó có thể dễ dàng mở rộng lên hai chiều để giải *Mobiles*; bộ nhớ chính cuối cùng cũng là một mảng tĩnh 4 MB, và hiệu quả cũng khá cao. Dùng cây tìm kiếm nhị phân để hiện thực có tư tưởng giống cây đoạn, vì hai cấu trúc này tương tự về bản chất.

Phương pháp này thường được dùng trong các bài toán thống kê sau rời rạc hóa, đặc biệt là các bài toán thống kê trên mặt phẳng.

3.3 Một ví dụ phức tạp hơn

Ví dụ 3: Khai mỏ (KOP). Một thợ mỏ lành nghề ở mỏ vàng sắp nghỉ hưu vào cuối năm. Để thưởng cho sự tận tụy của ông, giám đốc quyết định tặng ông một mảnh đất hình chữ nhật. Mảnh đất có chiều dài S và chiều rộng W . Người thợ có thể tự chọn mảnh đất này. Hiển nhiên mảnh đất chứa càng nhiều điểm khai vàng càng tốt. Nhiệm vụ của bạn là tính số điểm khai vàng nhiều nhất có thể nhận được. Nếu một điểm khai vàng nằm trên biên hình chữ nhật thì cũng phải được tính.

Nhiệm vụ. Đọc vị trí các điểm khai vàng và tính giá trị lớn nhất.

Dữ liệu vào. Dòng đầu của tệp KOP.IN là S và W ($1 \leq s, w \leq 10000$); chúng lần lượt song song với trục OX và trục OY , chỉ ra kích thước khu vực. Dòng tiếp theo là số nguyên n ($1 \leq n \leq 15000$), biểu thị tổng số điểm khai vàng. Sau đó là n dòng, mỗi dòng hai số nguyên, cho tọa độ một điểm khai vàng. Phạm vi tọa độ là $-30000 \leq x, y \leq 30000$.

Dữ liệu ra. Một số nguyên: số điểm khai vàng nhiều nhất.

Ví dụ vào.

```
1 2
12
0 0
1 1
2 2
```

3 3
4 5
5 5
4 2
1 4
0 5
5 0
2 3
3 2

Ví dụ ra.

4

Phân tích. Ví dụ trong đề tương ứng với một hình các điểm trên mặt phẳng. Đây là bài toán quét trên điểm. Để nghĩ đến rồi rắc hóa, chẳng hạn dùng hai đường thẳng quét sao cho vùng giữa hai đường có chênh lệch tọa độ X không vượt quá S . Sau đó thống kê mọi hình chữ nhật có chiều rộng W trong dải ấy:

S

L_1

L_2

Trong hình, L_1 và L_2 là hai đường quét. L_2 di chuyển phía trước L_1 ; bằng cách điều chỉnh để khoảng cách từ L_1 đến L_2 không lớn hơn S , các điểm trong dải giữa hai đường trở thành đối tượng nghiên cứu tiếp theo. Có thể thấy mỗi điểm vào và ra khỏi dải cần xử lý đúng một lần.

Với mọi điểm trong dải, vì chênh lệch hoành độ của chúng không lớn hơn S , ta có thể bỏ qua mọi hoành độ và chỉ xét tung độ. Ví dụ trong một dải có 5 điểm với tung độ $\{5, 3, 9, 1, 9\}$, $W = 2$. Tự nhiên ta nghĩ đến sắp các tọa độ thành $\{1, 3, 5, 9, 9\}$, rồi dùng một phương pháp quét tương tự trên hoành độ để tìm hình chữ nhật rộng W . Nhưng với bài này, cách đó không thật tốt. Xét kỹ hơn cách xử lý đặc biệt cho tung độ:

Với mỗi tọa độ y , tạo hai sự kiện điểm $(y, +1)$, $(y + W + 1, -1)$.

Ví dụ các điểm trên được đánh dấu thành

$(1, +1)$, $(4, -1)$, $(3, +1)$, $(6, -1)$, $(5, +1)$, $(8, -1)$,
 $(9, +1)$, $(12, -1)$, $(9, +1)$, $(12, -1)$.

Sắp chúng theo tọa độ y , ta được

$(1, +1)$, $(3, +1)$, $(4, -1)$, $(5, +1)$, $(6, -1)$, $(8, -1)$,
 $(9, +1)$, $(9, +1)$, $(12, -1)$, $(12, -1)$.

Ta phản ánh các nhãn phía sau lên một ánh xạ tọa độ y , rồi lấy tổng từ thấp đến cao:

Gia tri su kien: +1 +1 -1 +1 -1 -1 +2 -2
Tong tien to: 1 2 1 2 1 0 2 0

Chú ý các tổng dưới tọa độ: giá trị lớn nhất trong các tổng ấy chính là số điểm nhiều nhất mà một hình chữ nhật trong dải này có thể chứa. Tổng tại mỗi vị trí ghi tổng mọi nhãn trước vị trí đó.

Nhờ bước chuyển đổi khéo léo này, ta có thể dùng cây tìm kiếm nhị phân ở trên để xử lý các sự kiện điểm theo tọa độ y . Đồng thời như đã nói, mỗi điểm vào và ra khỏi dải đúng một lần; vì vậy ta phải tận dụng thống kê trên cây để sau khi chèn hoặc xóa một sự kiện điểm có thể duy trì các giá trị tổng $\sum$ dưới tọa độ, và có thể lấy giá trị lớn nhất trong các tổng ấy trong thời gian rất ngắn.

Thuật toán đại cường như sau. Ánh xạ mọi sự kiện điểm lên tọa độ y ; nhiều nhất có $n = 15000$ điểm, nên có thể có 30000 tọa độ khác nhau. Dùng các giá trị này xây một cây tìm kiếm nhị phân có thể thống kê, tức BUILD.

Trên mỗi nút của cây, đặt hai giá trị. Một là SUM, ghi tổng mọi giá trị trên các điểm thuộc cây con gốc tại nút đó, ban đầu $SUM = 0$. Hai là MAXSUM, ghi tổng $\sum$ lớn nhất trên cây con gốc tại nút đó; chú ý định nghĩa này lấy nút hiện tại làm gốc, tức chỉ xét giá trị trong cây con. Hàm sau có thể chèn một sự kiện điểm và duy trì tính chất của SUM và MAXSUM. Trong đó k là một nhãn, giá trị $+1$ hoặc -1 .

```
procedure INSERT( $y, k$ )
begin
  now <- 1
  repeat
    if  $V[now] = y$  then break
    if  $V[now] < y$  then  $now \leftarrow now * 2 + 1$ 
    else  $now \leftarrow now * 2$ 
  until false

  repeat
     $SUM[now] \leftarrow SUM[now] + k$ 
     $MAXSUM[now] \leftarrow \text{Max}(\text{MAXSUM}[now * 2],$ 
       $SUM[now] - SUM[now * 2 + 1],$ 
       $SUM[now] - SUM[now * 2 + 1] + MAXSUM[now * 2 + 1])$ 
     $now \leftarrow now \text{ div } 2$ 
  until  $now = 0$ 
end
```

Phân tích kỹ quá trình này, nó có hai phần: trước hết đi xuống để tìm nút chứa y , sau đó đi ngược lên và sửa các lượng liên quan trên từng điểm thuộc đường đi. Cách tính SUM không cần giải thích. MAXSUM thực chất là một quá trình quy hoạch động. Vì cần lấy giá trị lớn nhất của một tổng liên tiếp, có ba trường hợp: giá trị lớn nhất nằm trong cây con trái; giá trị lớn nhất vừa đúng chứa nút gốc; giá trị lớn nhất nằm trong cây con phải. Ba hạng tử trong hàm Max tương ứng với giá trị ở ba trường hợp ấy.

Sau khi hoàn thành thao tác INSERT, $MAXSUM[1]$ chính là một đáp án tối ưu hiện tại. Độ phức tạp thực hiện INSERT là chiều cao cây, tức $\log n$.

Hàm INSERT là phần cốt lõi để giải bài này. Có quá trình đó rồi thì dễ hoàn thành toàn bộ thuật toán: trước hết sắp mọi điểm theo hoành độ, sau đó dùng hai đường quét L_1 và L_2 . Theo phân tích ở trên, n điểm vào và ra khỏi dải quét mỗi loại một lần; mỗi lần tương ứng hai sự kiện điểm, nên tổng cộng có $4n$ thao tác INSERT. Độ phức tạp thời gian toàn thuật toán là $O(n \log n)$. Việc lập trình hiện thực cũng không phức tạp.

Qua ví dụ này có thể cảm nhận sâu sắc lợi ích của cây nhị phân. Cây nhị phân và cây đoạn ở phần trước tương tự về bản chất, chỉ là cách hiện thực này tiện và ngắn gọn hơn. Tất nhiên, với loại bài này trước hết phải chuyển đổi bài toán để có thể dùng cây nhị phân một cách hiệu quả. Trong ví dụ này chính nhờ khái niệm sự kiện điểm mà ta tìm được thuật toán hiệu quả.

4 Cây nhị phân ảo

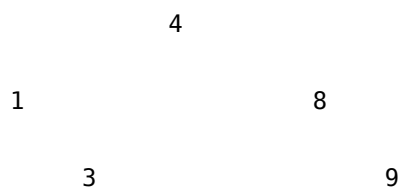
Từ phần ba có thể thấy cây nhị phân là một phương pháp xử lý tính rất thực dụng. Nhưng trước khi dùng, phải xây một cây nhị phân rỗng. Thực ra kết hợp với phương pháp chia đôi, ta có thể không cần xây riêng cấu trúc cây nhị phân. Phần này giới thiệu cách bỏ qua việc xây dựng ấy. Vì mất đi cấu trúc cây nhị phân hữu hình, ở đây ta gọi nó là hiện thực bằng cây nhị phân ảo.

4.1 Hình thái hiện thực cây nhị phân ảo

Tuy không xây cây này, ta tất nhiên vẫn phải dùng tính chất cân bằng của cây và sự tiện lợi trong tính toán. Ý tưởng ở đây bắt nguồn từ nhận xét: với bất kỳ bảng có thứ tự nào, khi tìm kiếm nhị phân trên đó, thực chất ta đang tìm trên một cây nhị phân. Ở đây vẫn giả sử đối tượng cần tìm luôn tồn tại trong bảng, ví dụ đoạn chương trình tìm kiếm nhị phân sau:

```
function BinSearch(x): integer
begin
  l <- 1
  r <- n
  while l <= r do
    begin
      m <- (l + r) div 2
      if V[m] = x then return m
      if V[m] > x then r <- m - 1
      else l <- m + 1
    end
  end
end
```

Mỗi lần m là điểm giữa của một khoảng $[l, r]$, điều này gần giống cách xây cây đoạn hoặc cây nhị phân đã nói ở trên. Với bảng $\{1, 3, 4, 8, 9\}$, tìm kiếm nhị phân tương đương tìm kiếm trên cây tìm kiếm nhị phân sau:



Nói theo cách khác, m lấy giá trị giữa, tức là gốc của một cây; mọi phần tử bên trái nó, tức $[l, m - 1]$, tạo thành hình thái trên cây con trái, còn $[m + 1, r]$ tạo thành hình thái trên cây con phải.

Tuy cây này không gần đây và không có cấu trúc đẹp như cây xây trong phần ba, nó vẫn là một cây nhị phân cân bằng. Chỉ từ tính chất của tìm kiếm nhị phân cũng biết rằng nhiều nhất $\log n$ lần tìm là có thể tìm được nút, tức việc sửa một giá trị cố định chỉ liên quan đến $\log n$ nút.

Trong phần ba, ta xây cây trước rồi sửa trên từng nút. Lợi ích của tư tưởng đó là có thể dùng chỉ số để lấy quan hệ cha con, cấu trúc rõ ràng. Trong phần này, qua khảo sát tìm kiếm nhị phân, thực ra ta rút ra rằng một bảng tuyến tính đã sắp thứ tự tương ứng với một cây. Hình thái của cây này không được xây thật, nhưng trong quá trình tìm kiếm nhị phân nó là cố định.

Trong phần ba, ta giữ mọi lượng cần sửa tại nút gốc. Chuyển phương pháp này sang phần này, gốc của mỗi cây con thực ra là giá trị ứng với m trong tìm kiếm nhị phân. Nhờ vậy ta bỏ được bước xây dựng.

Lấy ví dụ chèn nút để giải thích phương pháp hiện thực ảo này. Đặt LESS là số nút trên gốc và cây con trái:

```
procedure INSERT(x)
begin
  l <- 1
  r <- n
  while l <= r do
    begin
      m <- (l + r) div 2
      if x <= V[m] then LESS[m] <- LESS[m] + 1
      if x = V[m] then break
      if x < V[m] then r <- m - 1
      else l <- m + 1
    end
  end
end
```

Chỉ cần tưởng tượng trong đầu một cây tìm kiếm nhị phân ảo là không khó hiểu ý nghĩa cụ thể của quá trình này.

Hiện thực ảo thực chất bỏ qua việc xây dựng, nên nó đơn giản hơn nữa khi lập trình. Về hiệu quả, ta vẫn bảo đảm độ phức tạp không vượt quá $\log n$.

Trong hiện thực ảo, V thực ra chính là một bảng ánh xạ có thứ tự. Nó vẫn xử lý các bài toán có tính chất duy trì động; trước khi hiện thực, trước hết vẫn sắp thứ tự mọi đối tượng cần xử lý. Điểm này tương tự các phương pháp xử lý bài toán ở trên.

4.2 Một ví dụ cụ thể

Ví dụ 4. Trên một hệ tọa độ Descartes phẳng có n điểm, n nhiều nhất là 15000. Yêu cầu với mỗi điểm, tính có bao nhiêu điểm nằm ở phía dưới bên trái của nó; tức với mỗi điểm (x_i, y_i) , tìm số chỉ số j thỏa $x_j \leq x_i$ và $y_j \leq y_i$.

Phân tích. Ta đã thảo luận nhiều lần các bài toán có hình thức tương tự. Bây giờ ta dùng hiện thực ảo để giải bài này với độ phức tạp thời gian $O(n \log n)$.

Trước hết sắp thứ tự các điểm, với quy tắc ưu tiên x , và khi x bằng nhau thì ưu tiên y . Nhờ vậy với mọi $(x_i \leq x_j, y_i \leq y_j)$, có $i \leq j$. Sau đó ta không cần xét hoành độ nữa, vì trong bảng đã sắp, x luôn có thứ tự; ta chỉ khảo sát quan hệ bao hàm theo y .

Sắp thứ tự các tọa độ y , xây quan hệ ánh xạ và đặt vào mảng V , trong đó tọa độ y tăng dần. Chỉ cần lần lượt chèn từng y vào mảng LESS dùng để đếm theo thứ tự điểm đã sắp, đồng thời nhận được giá trị phía dưới bên trái của một điểm. Đổi hàm ở trên thành:

```
function INSERT_AND_GET(y): integer
begin
  l <- 1
  r <- n
  ans <- 0
  while l <= r do
    begin
```



```

        m <- (l + r) div 2
        if y <= V[m] then LESS[m] <- LESS[m] + 1
        if y >= V[m] then ans <- ans + LESS[m]
        if y = V[m] then break
        if y < V[m] then r <- m - 1
        else l <- m + 1
    end
    return ans
end

```

Hàm này không cần giải thích nhiều; độ phức tạp của nó là $O(\log n)$.

4.3 Tối ưu quy hoạch động cho dãy đơn điệu dài nhất

Dãy đơn điệu dài nhất là bài toán kinh điển dùng quy hoạch động. Nay lấy việc tìm dãy giảm dài nhất, giảm nghiêm ngặt, làm ví dụ để giải thích cách giải nó trong $O(n \log n)$. Giả sử đối tượng cần xử lý là dãy $a[1..n]$. Toàn bộ thuật toán quy hoạch động được hiện thực như sau:

```

procedure longest_decreasing_subsequence
begin
    ans <- 0
    for i <- 1 to n do
        begin
            j <- 1
            k <- ans
            while j <= k do
                begin
                    m <- (j + k) div 2
                    if b[m] > a[i] then j <- m + 1
                    else k <- m - 1
                end
            end
            if j > ans then ans <- ans + 1
            b[j] <- a[i]
        end
    end
    return ans
end

```

Chương trình này rất ngắn gọn, nhưng bên trong có không ít điểm tinh tế. Để hiểu quá trình này, ta bắt đầu phân tích từ cách giải cơ bản nhất.

Trước hết, ta đều biết thuật toán tìm dãy giảm dài nhất:

$$\begin{aligned}
 a[0] &= \infty, \\
 M[0] &= 0, \\
 M[i] &= \max\{M[j] + 1 \mid 0 \leq j < i \text{ và } a[j] > a[i]\}, \\
 P[i] &= j \quad (j \text{ là giá trị đạt max trong công thức trên}), \\
 \text{ans} &= \max\{M[i]\}.
 \end{aligned}$$

Trong công thức này, P biểu thị quyết định. Xét riêng P , có thể có nhiều quyết định đều làm $M[i]$ đạt giá trị lớn nhất, và các quyết định này tương đương nhau. Vì vậy ta đương nhiên có thể đặt một ràng buộc đặc biệt cho P : trong mọi quyết định tương đương j , P chọn quyết định có $a[j]$ lớn nhất.

Cách chọn P không ảnh hưởng đến kết quả nhận được, nhưng ta muốn dùng P để giải thích vấn đề sau: với số thứ x , nó có thể tạo thành một dãy giảm dài nhất có độ dài $M[x]$; bài toán con của nó là một dãy giảm dài nhất độ dài $M[x] - 1$ trong $a[1..x - 1]$, và số cuối của dãy này lớn hơn $a[x]$. Ta để P chọn dãy có số cuối lớn nhất trong mọi lời giải khả dĩ ấy. Nói cách khác, sau khi xử lý xong $a[1..x - 1]$, với mọi dãy giảm độ dài $M[x] - 1$, quyết định $P[x]$ chỉ liên quan đến dãy có số cuối lớn nhất trong số đó; các dãy cùng độ dài còn lại không cần quan tâm.

Từ đó nghĩ đến việc dùng một mảng động khác b . Khi ta tính xong $a[x]$, mọi dãy giảm thu được trong $a[1..x]$ được chia theo độ dài thành L lớp tương đương; trong mỗi lớp chỉ cần một đại diện, và đại diện này có số cuối lớn nhất trong lớp. Ta ghi nó là $b[j]$, $1 \leq j \leq L$. $b[j]$ là đại diện của dãy có số cuối lớn nhất trong mọi dãy giảm độ dài j .

Vì các kết quả của $a[1..x - 1]$ đều được ghi trong b , khi xử lý số hiện tại $a[x]$, ta không cần so sánh với các $a[j]$ trước đó ($1 \leq j \leq x - 1$), mà chỉ cần so sánh với $b[j]$ ($1 \leq j \leq L$).

Với việc xử lý $a[x]$, ta giải thích ngắn gọn. Trước hết, nếu $a[x] < b[L]$, tức trong $a[1..x - 1]$ chỉ tồn tại các dãy giảm độ dài từ 1 đến L , và $b[L]$ là đại diện của dãy độ dài L . Vì $a[x] < b[L]$, hiển nhiên có thể nối $a[x]$ sau dãy này để tạo thành một dãy độ dài $L + 1$. $a[x]$ cũng có thể nối sau bất kỳ $b[j]$ nào ($1 \leq j < L$) để tạo thành dãy độ dài $j + 1$, nhưng chắc chắn $a[x] < b[j + 1]$, nên nó không thể trở thành đại diện của $b[j + 1]$. Khi đó $b[L + 1] = a[x]$, tức $a[x]$ là đại diện của dãy độ dài $L + 1$, đồng thời L phải tăng thêm 1.

Khả năng khác là $a[x] \geq b[1]$. Hiển nhiên khi đó $a[x]$ là phần tử lớn nhất trong $a[1..x]$; nó chỉ có thể tạo thành dãy giảm độ dài 1, đồng thời chắc chắn là lớn nhất, nên nó làm đại diện cho $b[1]$, tức $b[1] = a[x]$.

Nếu các trường hợp trên đều không xảy ra, chắc chắn ta tìm được một j , $2 \leq j \leq L$, sao cho $b[j - 1] > a[x]$ và $b[j] \leq a[x]$. Khi đó $a[x]$ tất nhiên nối sau $b[j - 1]$, tạo thành một dãy mới độ dài j . Lý do là nếu $a[x]$ nối sau bất kỳ $b[k]$ nào với $1 \leq k < j - 1$, đều có $b[k + 1] > a[x]$, nên $a[x]$ không thể làm đại diện. Còn với bất kỳ $b[k]$ nào mà $j \leq k \leq L$, $b[k] \leq a[x]$, nên $a[x]$ không thể kéo dài dãy đó. Vì $a[x] \geq b[j]$, ta cập nhật $b[j]$ thành $a[x]$.

Sau bất kỳ trường hợp nào, $b[1..L]$ hiển nhiên là một dãy giảm, nhưng nó không biểu thị một dãy giảm độ dài L ; điểm này không được nhầm.

Các trường hợp trên thực ra đều quy về: xử lý $a[x]$, đặt $b[L + 1]$ là âm vô cùng, từ trái sang phải tìm vị trí nhỏ nhất j sao cho $b[j] \leq a[x]$, rồi chỉ cần gán $b[j] = a[x]$; nếu $j = L + 1$ thì đồng thời tăng L . Độ dài dãy giảm dài nhất kết thúc tại vị trí x là $M[x] = j$.

Đoạn chương trình có thể mô tả trước như sau:

```
procedure longest_decreasing_subsequence_simple
begin
  L <- 0
  for x <- 1 to n do
    begin
      b[L + 1] <- -infinity
      j <- 1
      while b[j] > a[x] do j <- j + 1
      b[j] <- a[x]
      if j > L then L <- j
    end
  return L
end
```

Chú ý phần tìm tuyến tính trong đoạn chương trình này dễ suy biến; khi bản thân a là dãy giảm, nó suy biến thành thuật toán $O(n^2)$.

Lúc này ta cần dùng phương pháp chia đôi của phần này. Phần tìm tuyến tính là tìm j nhỏ nhất thỏa $b[j] \leq a[x]$. Vì $b[1..L]$ chắc chắn là một dãy đơn điệu giảm có thứ tự, ta chỉ cần dùng tìm kiếm nhị phân để tìm vị trí này. Nguyên lý của nó tương đương tìm kiếm trên cây nhị phân. Vì vậy ta có đoạn chương trình kinh điển đã nêu ở đầu. Phần then chốt là:

```
j <- 1
k <- ans
while j <= k do
begin
    m <- (j + k) div 2
    if b[m] > a[i] then j <- m + 1
    else k <- m - 1
end
if j > ans then ans <- ans + 1
b[j] <- a[i]
```

Trên đây đã giải quyết độ dài dãy giảm dài nhất. Thực ra để giải dãy tăng, hoặc dãy không tăng dài nhất, chỉ cần sửa nhẹ các dấu bất đẳng thức trong thuật toán. Sau khi hiểu nguyên lý của phương pháp này, việc sửa đổi ấy không khó.

Bài này còn có điểm thú vị: trong khi dùng b để tìm độ dài dãy giảm dài nhất, ta cũng hoàn thành việc dựng một phủ của dãy a bằng số dãy không giảm ít nhất. Nói cách khác, ta có thể dùng phương pháp này để chứng minh một mệnh đề thú vị: số phủ nhỏ nhất bằng các dãy không giảm của một dãy bằng độ dài dãy giảm dài nhất.

Vì sao nói ta đã hoàn thành việc dựng? Chỉ cần chú ý rằng mỗi lần xử lý $a[x]$, ta cập nhật $b[j]$ thành $a[x]$. Ý nghĩa dựng của nó là: $a[x]$ nối sau đại diện mà $b[j]$ đang biểu thị, tức điểm kế tiếp trên đường phủ của đại diện $b[j]$ là $a[x]$. Đồng thời khi L tăng, ta mở riêng một đường phủ mới, và $a[x]$ là nút đầu tiên trên đường này.

Có lẽ phần giải thích quy hoạch động của bài này hơi dài dòng. Nếu biến đổi từng bước các phương trình quy hoạch động thì cũng có thể thu được phương án cuối cùng. Trong bài viết này, ta cố gắng giải thích trực tiếp tính đúng đắn của nó, nên không dùng phương pháp gián tiếp.

Kết luận

Bài viết này xoay quanh việc giải các bài toán thống kê. Có thể thấy phần lớn các bài toán được thảo luận thuộc loại bài toán rời rạc hóa trên mặt phẳng. Tư tưởng cơ bản để giải các bài toán này thực chất là dùng phương pháp chia đôi và dùng cách hiện thực cấu trúc dữ liệu đặc biệt để nâng cao hiệu quả. Các phương pháp trong mấy phần trên về cơ bản tương tự nhau, nhưng mỗi phần thảo luận một cách hiện thực cấu trúc dữ liệu khác nhau, vì vậy cũng có thể nói bài viết này thiên về kỹ xảo.

Đặc biệt, có thể thấy các ví dụ trong phần một, ba và bốn đều có thể hiện thực bằng vài phương pháp; tất nhiên ta nên chọn phương pháp dễ và gọn hơn. Nói chung, cây đoạn thích hợp cho bài toán hình học. Các ví dụ cụ thể của nó được mô tả rất rõ trong tài liệu tham khảo liên quan; bài viết này chỉ giới thiệu kiến thức cơ bản ở phương diện ấy để tạo gợi ý nhất định. Phương pháp ở phần hai có thể nói là sự cô đọng của kỹ thuật cao hơn. Trọng tâm của bài viết nằm ở phần ba và bốn: hai phương pháp tính này thích hợp hơn với đa số bài toán thống kê điểm. Cuối cùng bổ sung một ví dụ quy hoạch động; nó thực ra

là một biểu hiện của hiện thực ảo, đồng thời nhắc ta rằng trong phần lớn bài toán có thứ tự, dùng phương pháp chia đôi có thể đem lại tác dụng then chốt.

Tài liệu tham khảo

1. Bài viết đội tuyển tập huấn Trung Quốc IOI 1999 của Chen Hong, *Lựa chọn cấu trúc dữ liệu và hiệu quả*.
2. *Hình học tính toán: phân tích và thiết kế thuật toán*, Nhà xuất bản Đại học Thanh Hoa.
3. *Thuật toán và cấu trúc dữ liệu*, ấn bản thứ hai, Nhà xuất bản Công nghiệp Điện tử.
4. Tài liệu *Dynamic Programming* của USACO Training Gateway.
5. Phân tích đề IOI 2001, ấn bản thứ hai.

A Bản trình chiếu kèm theo trong PDF

Sau phần bài viết, PDF còn chứa một bản trình chiếu khái quát các ý chính. Phần này dịch lại toàn bộ phần chữ trích xuất được từ các trang trình chiếu.

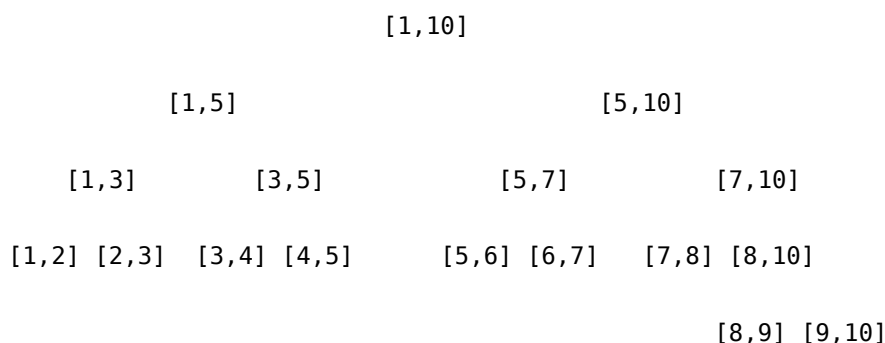
A.1 Giới thiệu

- Đặc điểm của các bài toán đếm trong một phạm vi nhất định:
- Mô tả đơn giản.
- Yêu cầu duy trì dữ liệu động và tính toán động.
- Công cụ giải quyết: khuôn mẫu và cấu trúc thống kê đặc biệt.

A.2 Cây đoạn

Cây đoạn xử lý động các đoạn cố định có thể ánh xạ lên một trục tọa độ, chẳng hạn tìm tổng độ dài của hợp các khoảng hoặc số khoảng trong hợp. Ưu điểm của nó là có thể tùy lúc chèn một khoảng hoặc xóa một khoảng đã có, đồng thời duy trì tính chất cần thiết với chi phí thấp.

Một cây đoạn biểu diễn $[1, 10]$:



Cấu trúc động:

Type

```
Tnode = ^Treenode;
Treenode = record
    B, E: integer;
    Count: integer;
    LeftChild, RightChild: Tnode;
End;
```

Trong đó B, E biểu thị khoảng $[B, E]$, Count thường ghi số đoạn phủ đến khoảng này, còn hai con trỏ chỉ đến gốc cây con trái và phải.

Cấu trúc tĩnh dùng các mảng $B[], E[], C[], LSON[], RSON[]$. Với gốc v , $B[v]$ và $E[v]$ là biên khoảng, $C[v]$ là bộ đếm, còn $LSON[v]$, $RSON[v]$ là số hiệu con trái và con phải.

Xây cây: bắt đầu từ khoảng gốc $[a, b]$, mỗi lần chia thành hai phần và xây các cây con tương ứng cho đến khi gặp khoảng sơ cấp $[x, x + 1]$.

Chèn khoảng $[c, d]$ vào cây đoạn $T(a, b)$, với số hiệu gốc là v :

```
procedure INSERT(c, d; v)
begin
    mid <- (B[v] + E[v]) div 2
    if c <= B[v] and E[v] <= d then C[v] <- C[v] + 1
    else
        begin
            if c < mid then INSERT(c, d; LSON[v])
            if d > mid then INSERT(c, d; RSON[v])
        end
    end
end
```

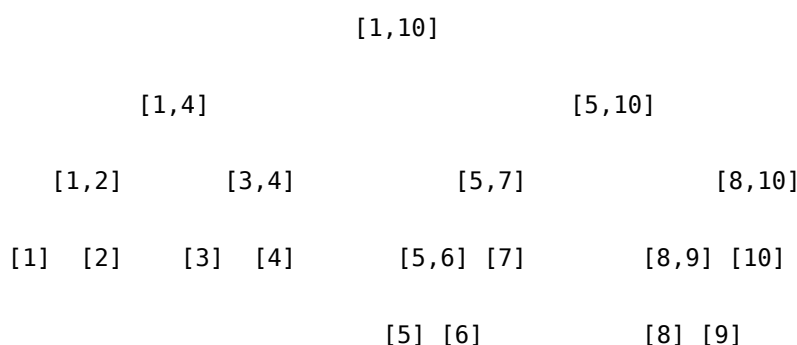
Ví dụ một tính chất đặc biệt: muốn lấy độ dài hợp các đoạn trên cây đoạn, thêm trường $M[v]$:

$$M[v] = E[v] - B[v] \quad (C[v] > 0),$$

$$M[v] = 0 \quad (C[v] = 0 \text{ và } v \text{ là lá}),$$

$$M[v] = M[LSON[v]] + M[RSON[v]] \quad (C[v] = 0 \text{ và } v \text{ là nút trong}).$$

Dạng biến đổi để thống kê điểm:



Ví dụ *Promotion*: xây cây đoạn điểm trên phạm vi $[1, 1\,000\,000]$, tức mỗi mệnh giá hóa đơn là một lá. Nếu $C[LSON[v]] > 0$ thì giá trị nhỏ nhất trong cây v nằm ở cây con trái. Nếu $C[RSON[v]] > 0$ thì giá trị lớn nhất nằm ở cây con phải.

A.3 Một phương pháp thống kê tĩnh

Ví dụ *Mobiles* của IOI 2001: trong lưới $N \times N$, ban đầu mỗi ô bằng 0. Cần xử lý động các câu hỏi tổng hình chữ nhật con $(x_1, y_1) - (x_2, y_2)$ và các phép sửa một ô (x, y) bằng cách cộng hoặc trừ một giá trị A . $1 \leq N \leq 1024$, tổng số câu hỏi và sửa đổi có thể đạt 60000.

Bài toán tổng dãy một chiều: các phần tử lưu trong $a[]$, chỉ số $1..n$, cần cập nhật động $a[x]$ và tính tổng từ $a[x_1]$ đến $a[y_1]$.

Với $a[1..n]$, đặt mảng $C[1..n]$:

$$C[i] = a[i - 2^k + 1] + \dots + a[i],$$

trong đó k là số bit 0 ở cuối i trong nhị phân. Ví dụ 1001010110010000_2 có $k = 4$. Tính

$$\text{LOWBIT}(x) = x \text{ and } (x \text{ xor } (x - 1)).$$

Sửa $a[x]$ liên quan đến những C nào? Ví dụ $x = 76 = 1001010_2$:

$$p_1 = 1001010,$$

$$p_2 = 1001100,$$

$$p_3 = 1010000,$$

$$p_4 = 1100000,$$

$$p_5 = 10000000.$$

Các vị trí này thỏa

$$p_1 = x, \quad p_i < p_{i+1}, \quad p_{i+1} = p_i + \text{LOWBIT}(p_i).$$

Ta sửa $C[p_1], C[p_2], \dots$

procedure UPDATA(x, A)

begin

$p \leftarrow x$

 while $p \leq n$ do

 begin

$C[p] \leftarrow C[p] + A$

$p \leftarrow p + \text{LOWBIT}(p)$

 end

end

Chi phí duy trì là $\log n$. Tính tổng $a[1]$ đến $a[x]$:

function SUM(x)

begin

$\text{ans} \leftarrow 0$

$p \leftarrow x$

 while $p > 0$ do

 begin

$\text{ans} \leftarrow \text{ans} + C[p]$

$p \leftarrow p - \text{LOWBIT}(p)$

 end

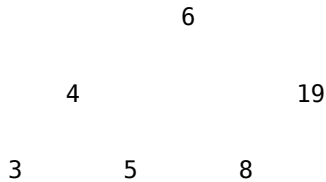
 return ans

end

Mở rộng về bài toán hai chiều ban đầu bằng cách xây $C[x, y]$, mỗi chiều có định nghĩa giống một chiều. Sau khi mở rộng, thuật toán có hai lớp lồng nhau, chi phí một lần duyệt là $O(\log^2 n)$.

A.4 Hiện thực bằng cây tìm kiếm nhị phân tĩnh

Với tập $\{3, 4, 5, 8, 19, 6\}$, có thể xây cây:



Cách xây dựng đệ quy: xây ánh xạ tọa độ điểm X , đặt $p \leftarrow 0$ là con trỏ trong X , rồi:

```

procedure BUILD(ID: integer)
begin
  if ID * 2 <= n then BUILD(ID * 2)
  p <- p + 1
  V[ID] <- X[p]
  if ID * 2 + 1 <= n then BUILD(ID * 2 + 1)
end
  
```

{Trong chương trình chính gọi BUILD(1).}

Cách xây dựng không đệ quy: lần lượt tìm chỉ số của điểm kế tiếp. Điểm đầu tiên *first* chắc chắn là vị trí ngoài cùng bên trái ở tầng dưới cùng. Nếu n điểm có L tầng thì $\text{first} = 2^{L-1}$. Nếu vị trí hiện tại là *now*: nếu có con phải, tức $\text{now} * 2 + 1 \leq n$, vị trí kế tiếp là điểm ngoài cùng bên trái ở dưới cây con phải; nếu không có con phải, khi *now* là số lẻ thì chia *now* cho 2 cho đến khi *now* là số chẵn, cuối cùng lại chia *now* cho 2.

Chèn một điểm x :

```

procedure INSERT(x)
begin
  now <- 1
  repeat
    SUM[now] <- SUM[now] + 1
    if V[now] = x then break
    if V[now] > x then now <- now * 2
    else now <- now * 2 + 1
  until false
end
  
```

SUM ghi tổng số nút trên một cây con; cách xóa tương tự.

Giải ví dụ 2 bằng LESS, là tổng trọng số trên gốc và cây con trái:

```

procedure INSERT2(x, A)
begin
  
```

```

now <- 1
repeat
  if x <= V[now] then LESS[now] <- LESS[now] + A
  if V[now] = x then break
  if V[now] > x then now <- now * 2
  else now <- now * 2 + 1
until false
end

```

Tính tổng $a[1..x]$:

```

function SUM(x): longint
begin
  ans <- 0
  now <- 1
  repeat
    if V[now] <= x then ans <- ans + LESS[now]
    if V[now] = x then break
    if V[now] > x then now <- now * 2
    else now <- now * 2 + 1
  until false
  return ans
end

```

Ví dụ khai mỏ (KOP). Một thợ mỏ sắp nghỉ hưu được tặng một mảnh đất hình chữ nhật dài S , rộng W . Ông có thể tự chọn vị trí mảnh đất. Cần tính số điểm khai vàng nhiều nhất nằm trong mảnh đất; điểm trên biên cũng được tính. Dữ liệu gồm S, W , số điểm n , rồi n cặp tọa độ. Kết quả là số điểm nhiều nhất.

Quét theo X bằng hai đường L_1, L_2 :

S

L1 L2

Với mỗi tọa độ y , tạo hai sự kiện điểm $(y, +1)$ và $(y + W + 1, -1)$. Ví dụ trong một dải có 5 điểm với tung độ $\{5, 3, 9, 1, 9\}$, $W = 2$, ta tạo

$(1, +1), (4, -1), (3, +1), (6, -1), (5, +1), (8, -1),$
 $(9, +1), (12, -1), (9, +1), (12, -1).$

Sắp theo tọa độ y rồi lấy tổng từ thấp lên cao:

Gia trị sự kiện:	+1	+1	-1	+1	-1	-1	+2	-2
Tổng tiền tố:	1	2	1	2	1	0	2	0

SUM[now] là tổng mọi nhãn $+1, -1$ trên cây con. MAXSUM[now] là giá trị tiền tố lớn nhất trong cây con; MAXSUM[1] là đáp án tốt nhất ở một trạng thái. Có ba khả năng: giá trị lớn nhất ở cây con trái, vừa đúng chứa gốc, hoặc ở cây con phải. Duyệt từ dưới lên:


```

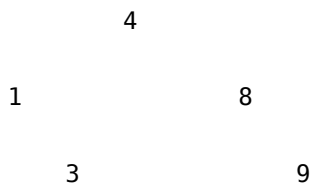
repeat
  SUM[now] <- SUM[now] + k
  MAXSUM[now] <- Max(
    MAXSUM[now * 2],
    SUM[now] - SUM[now * 2 + 1],
    SUM[now] - SUM[now * 2 + 1] + MAXSUM[now * 2 + 1])
  now <- now div 2
until now = 0

```

A.5 Hiện thực cây bằng phương pháp chia đôi ảo

Trước khi dùng cây nhị phân, thường phải xây một cây rỗng. Nhưng với bất kỳ bảng có thứ tự nào, tìm kiếm nhị phân trên bảng thực chất tương đương tìm kiếm trên một cây nhị phân.

Với bảng {1, 3, 4, 8, 9}, tìm kiếm nhị phân tương đương cây:



Ví dụ chèn nút, đặt LESS là số nút trên gốc và cây con trái:

```

procedure INSERT(x)
begin
  l <- 1
  r <- n
  while l <= r do
    begin
      m <- (l + r) div 2
      if x <= V[m] then LESS[m] <- LESS[m] + 1
      if x = V[m] then break
      if x < V[m] then r <- m - 1
      else l <- m + 1
    end
  end
end

```

A.6 Ví dụ trong trình chiếu: dãy giảm dài nhất

Cho một dãy số nguyên a , tìm độ dài dãy giảm dài nhất. Công thức quy hoạch động cơ bản:

$$\begin{aligned}
 a[0] &= \infty, \\
 M[0] &= 0, \\
 M[i] &= \max\{M[j] + 1 \mid 0 \leq j < i \text{ và } a[j] > a[i]\}, \\
 P[i] &= j \quad (j \text{ là vị trí đạt max}), \\
 \text{ans} &= \max\{M[i]\}.
 \end{aligned}$$

Độ phức tạp là $O(n^2)$.

Đặt hạn chế đặc biệt cho P : trong mọi quyết định tương đương j , chọn $a[j]$ lớn nhất. Sau khi xử lý xong $a[1..x - 1]$, với mọi dãy giảm độ dài $M[x] - 1$, quyết định $P[x]$ chỉ liên quan đến dãy có phần tử cuối lớn nhất. Dùng mảng động b : khi tính xong $a[x]$, các dãy giảm trong $a[1..x]$ chia theo độ dài thành L lớp; mỗi lớp chỉ cần đại diện có phần tử cuối lớn nhất, ghi là $b[j]$, $1 \leq j \leq L$. Khi xử lý $a[x]$, không cần so sánh với mọi phần tử trước, chỉ cần so sánh với $b[j]$.

Nếu $a[x] < b[L]$, nối $a[x]$ sau dãy đại diện độ dài L , tạo dãy độ dài $L + 1$, đặt $b[L + 1] = a[x]$ và tăng L . Nếu $a[x] \geq b[1]$, $a[x]$ chỉ có thể tạo dãy độ dài 1, đồng thời là đại diện mới của $b[1]$. Nếu không, tìm j sao cho $b[j - 1] > a[x]$ và $b[j] \leq a[x]$, rồi đặt $b[j] = a[x]$.

Các trường hợp này quy về: xử lý $a[x]$, đặt $b[L + 1]$ là âm vô cùng, từ trái sang phải tìm vị trí đầu tiên j sao cho $b[j] \leq a[x]$, rồi gán $b[j] = a[x]$; nếu $j = L + 1$, tăng L .

```
L <- 0
for x <- 1 to n do
begin
  b[L + 1] <- -infinity
  j <- 1
  while b[j] > a[x] do j <- j + 1
  b[j] <- a[x]
  if j > L then L <- j
end
```

Đổi thành tìm kiếm nhị phân:

```
j <- 1
k <- L
while j <= k do
begin
  m <- (j + k) div 2
  if b[m] > a[i] then j <- m + 1
  else k <- m - 1
end
if j > L then L <- L + 1
b[j] <- a[i]
```

Tóm lại: rời rạc hóa; chuyển đổi bài toán; xây cấu trúc có thể thống kê.